CANVIA

REST ASSURED + CUCUMBER · SESIÓN 3 DE 4

Código que escala y caza de bugs

Especificaciones reutilizables, un cliente por recurso, URL por ambiente, log solo cuando falla, etiquetas, reportes que cualquiera entiende y una API que esconde al menos cinco bugs.

Instructor **Giancarlo Palomino** Jueves 29 de octubre · 15:00 a 17:00 Entrega: **feature/s3-<tu-nombre>**

HOY · 120 MINUTOS

Primero ordenamos la casa, después cazamos

HORA	MIN	QUÉ HACEMOS
15:00 – 15:35	35	`RequestSpecification`, cliente de API por recurso, URL por ambiente, log solo cuando falla
15:35 – 15:50	15	Esquema del escenario con casos negativos y etiquetas (`@smoke` , `@regresion`)
15:50 – 16:10	20	Reportes de Cucumber: HTML, JSON y JUnit; cómo leer un fallo
16:10 – 17:00	50	**Ejercicio 5 · Caza de bugs**: la API esconde al menos cinco

5

Ejercicio 5

Escenarios que describen lo correcto, hoy fallan y llevan `@bug` . Más un `BUGS.md` : qué hiciste, qué esperabas, qué pasó.

PARTIMOS DE

Los ejercicios 3 y 4 en verde: token, PUT, PATCH, DELETE, records y JSON Schema.

LA IDEA DEL DÍA

Si mañana cambia la URL, una cabecera o el ambiente, se toca **un** archivo. Ningún `.feature` .

EL SÍNTOMA

Cuando cada step arma su propia petición

LO QUE VIMOS EN LOS PR

Cada método repite `given().contentType(ContentType.JSON).accept("application/json")`. Veinte steps, veinte copias.

RUTAS SUELTAS

`"/booking/{id}"` escrito en seis lugares. Si la API pasa a `/v2/booking`, hay que buscar y reemplazar.

LA URL FIJA Y EL LOG DE TODO

Un único ambiente escrito en el código, y `.log().all()` en cada petición: la consola se llena y el fallo real se pierde.

LA CURA

Cada cosa en **un** lugar. Los steps no saben de HTTP; el cliente no sabe de Gherkin.

CAPA 1 · QUÉ

.feature

negocio en español, sin técnica

CAPA 2 · TRADUCIR

Steps

frase → llamada al cliente → verificación

CAPA 3 · CÓMO

ReservaCliente · AutenticacionCliente

rutas, verbos, token

CAPA 4 · LO COMÚN

Especificaciones.base()

URL, cabeceras, log

REQUESTSPECIFICATION

Lo que toda petición comparte, escrito una vez

api/Especificaciones.java · base() JAVA

```java
public static RequestSpecification base() {
    return new RequestSpecBuilder()
            .setBaseUri(Configuracion.baseUrl())    // por ambiente: -DbaseUrl=...
            .setContentType(ContentType.JSON)       // lo que enviamos
            .setAccept("application/json")          // lo que pedimos (evita el
418)
            // log de petición y respuesta, solo si una validación falla
            .setConfig(RestAssuredConfig.config().logConfig(LogConfig.logConfig()
                    .enableLoggingOfRequestAndResponseIfValidationFails()))
            .build();
}
```

CÓMO SE USA

`given().spec(Especificaciones.base())` y luego solo lo propio de esa petición: cuerpo, parámetros, token.

UN MÉTODO, NO UNA CONSTANTE

Cada llamada construye una especificación nueva. Así un step no "ensucia" la de otro con una cookie o un parámetro.

¿Y RESTASSURED.BASEURI?

Funciona, pero es global y estático: lo que cambie un escenario lo ve el siguiente. La especificación es explícita.

UN CLIENTE POR RECURSO

ReservaCliente: el único que sabe de /booking

api/ReservaCliente.java — JAVA

```java
public class ReservaCliente {

    /** Acepta un record Reserva o un Map (para enviar cuerpos inválidos). */
    public Response crear(Object cuerpo) {
        return given().spec(Especificaciones.base())
                .body(cuerpo)
                .when().post("/booking");
    }

    public Response consultar(int id) {
        return given().spec(Especificaciones.base())
                .pathParam("id", id)
                .when().get("/booking/{id}");
    }

    public Response buscarPorApellido(String apellido) {
        return given().spec(Especificaciones.base())
                .queryParam("lastname", apellido)
                .when().get("/booking");
    }
    // ... ping, actualizar, actualizarParcial, eliminar y conToken: igual que
ayer
}
```

DEVUELVE RESPONSE, NO VERIFICA

El cliente hace la petición; el step decide qué esperar. Así el mismo `eliminar` sirve para el caso feliz (201) y para el negativo (403).

NOMBRES DE NEGOCIO

`buscarPorApellido`, no `getBookingWithQuery`. El step se lee casi como el Gherkin.

EN EL EXAMEN

Canvia Market tiene productos y pedidos: un `ProductoCliente` y un `PedidoCliente`, con la misma forma.

PICOCONTAINER

Nadie hace new: el contenedor reparte las piezas

steps/ReservaSteps.java — JAVA

```java
public class ReservaSteps {

    private final Contexto contexto;
    private final ReservaCliente reservas;

    // PicoContainer crea el Contexto y el cliente, y los inyecta aquí.
    public ReservaSteps(Contexto contexto, ReservaCliente reservas) {
        this.contexto = contexto;
        this.reservas = reservas;
    }

    @Cuando("busco reservas con el apellido {string}")
    public void buscoPorApellido(String apellido) {
        contexto.respuesta = reservas.buscarPorApellido(apellido);
    }
}
```

support/Hooks.java — JAVA

```java
@After("@limpieza")
public void borrarReservasCreadas() {
    if (contexto.reservasCreadas.isEmpty()) {
        return;
    }
    String token = contexto.token != null
            ? contexto.token
            : autenticacion.obtenerToken(Configuracion.usuario(),
Configuracion.clave());
    contexto.reservasCreadas.forEach(id -> reservas.eliminar(id, token));
}
```

UNA INSTANCIA POR ESCENARIO

Steps, hooks y clientes reciben el **mismo** `Contexto` durante un escenario. El siguiente escenario empieza con todo nuevo.

PREGUNTA PARA LA SALA

¿Por qué el hook de limpieza no verifica el código de respuesta del `DELETE`?

URL POR AMBIENTE

El mismo código contra QA, staging o tu PC

pom.xml — XML

```xml
<baseUrl>https://restful-booker.herokuapp.com</baseUrl>
...
<systemPropertyVariables>
  <baseUrl>${baseUrl}</baseUrl>
  <api.usuario>${api.usuario}</api.usuario>
  <api.clave>${api.clave}</api.clave>
</systemPropertyVariables>
```

support/Configuracion.java — JAVA

```java
public static String baseUrl() {
    return valor("baseUrl", "BASE_URL");
}
```

QUIERO…	COMANDO
El ambiente por defecto	`mvn test`
Otro ambiente	`mvn test "-DbaseUrl=https://qa.miapi.com"`
Desde la CI	variable `BASE_URL` en el job
El portal del examen	`"-DbaseUrl=http://localhost:3000"`

PRUÉBALO Y MÍRALO FALLAR

Con `"-DbaseUrl=https://restful-booker.herokuapp.com/noexiste"` el log muestra *Request URI: …/noexiste/ping* y *Expected status code <201> but was <404>*. Prueba de que la URL viene de afuera.

LOG SOLO CUANDO FALLA

Silencio si todo va bien, evidencia completa si algo se rompe

LA CONFIGURACIÓN

`enableLoggingOfRequestAndResponseIfValidationFails()` en la especificación. REST Assured guarda la petición y la respuesta y las imprime solo si un `then()` falla.

CORRIDA EN VERDE

23 escenarios, 0 líneas de *Request method* en la consola.

CORRIDA EN ROJO

Justo encima del ✗ aparece qué se envió, a dónde, con qué cabeceras, y qué respondió la API. Todo lo que pide un reporte de bug.

```
Esquema del escenario: BUG-01 · Una reserva con precio negativo se rechaza
  ✓ Cuando intento crear una reserva con "precio negativo"
Request method: POST
Request URI:    https://restful-booker.herokuapp.com/booking
Headers:        Accept=application/json
                Content-Type=application/json
Body:
{
    "firstname": "Bruno",
    "totalprice": -100,
    ...
}

HTTP/1.1 200 OK
{
    "bookingid": 1205,
    "booking": { "firstname": "Bruno", "totalprice": -100, ... }
}
  ✗ Entonces el código de respuesta es 400
      java.lang.AssertionError: 1 expectation failed.
      Expected status code <400> but was <200>.
```

02

Casos negativos y etiquetas

Una tabla de ejemplos para todo lo que la API debe rechazar, y etiquetas para decidir qué corre y cuándo.

CASOS NEGATIVOS

Una regla, muchos intentos de romperla

features/validaciones.feature — GHERKIN

```
@seguridad
Esquema del escenario: Cambiar una reserva exige autenticación
(<operación>)
  Dado que existe una reserva a nombre de "Pedro"
  Cuando intento <operación> esa reserva sin autenticarme
  Entonces el código de respuesta es 403

  Ejemplos:
    | operación  |
    | actualizar |
    | modificar  |
    | eliminar   |

Esquema del escenario: Pedir una reserva que no existe (<caso>)
  Cuando consulto la reserva con id <id>
  Entonces el código de respuesta es 404

  Ejemplos:
    | caso                | id        |
    | id cero             | 0         |
    | id negativo         | -1        |
    | id que nunca existió | 999999999 |
```

steps/ReservaSteps.java — JAVA

```
@Cuando("intento {word} esa reserva sin autenticarme")
public void intentoSinToken(String operacion) {
    int id = contexto.idReserva;
    Reserva reserva = contexto.reservaEnviada;
    Map<String, Object> cambio = Map.of("additionalneeds", "Nada");
    contexto.respuesta = switch (operacion) {
        case "actualizar" -> reservas.actualizar(id, reserva, null);
        case "modificar" -> reservas.actualizarParcial(id, cambio, null);
        case "eliminar" -> reservas.eliminar(id, null);
        default -> throw new IllegalArgumentException("Operación desconocida: " +
operacion);
    };
}
```

EL CASO EN EL TÍTULO

`(<operación>)` en el nombre hace que el reporte diga *… exige autenticación (eliminar)* en vez de *Ejemplo #1.3*.

UNA COLUMNA QUE DOCUMENTA

`caso` no se usa en ningún paso: explica **por qué** existe cada fila.

QUÉ CORRE Y CUÁNDO

Una estrategia de etiquetas que cabe en una tabla

ETIQUETA	SIGNIFICA	CUÁNDO CORRE
`@smoke`	Lo mínimo que debe funcionar (3 escenarios)	En cada Pull Request
`@regresion`	Todo lo que hoy debe estar en verde	Push a `develop` y cada mañana
`@bug`	Describe lo correcto y hoy falla	Solo a pedido
`@seguridad` · `@contrato` · `@negativo`	Temas, para filtrar	A pedido
`@ejercicio1` … `@ejercicio5`	Para revisar una entrega	A pedido
`@limpieza`	Activa el hook que borra lo creado	No es filtro: es comportamiento

src/test/resources/junit-platform.properties — PROPS

```
cucumber.publish.quiet=true
cucumber.filter.tags=not @bug
```

ETIQUETA EN LA CARACTERÍSTICA

`@regresion` arriba de `Característica:` la heredan todos sus escenarios.

POR QUÉ EN EL .PROPERTIES Y NO EN EL RUNNER

Un `-Dcucumber.filter.tags=…` gana al archivo; una anotación del runner, no. Así el valor por defecto se puede cambiar desde la consola y la CI.

REPORTES DE CUCUMBER

Tres reportes, tres lectores

runners/RunCucumberTest.java — JAVA

```java
@Suite
@IncludeEngines("cucumber")
@SelectPackages("features")
@ConfigurationParameter(key = GLUE_PROPERTY_NAME, value =
"com.canvia.qa")
@ConfigurationParameter(key = PLUGIN_PROPERTY_NAME, value = "pretty, "
        + "html:target/cucumber-report.html, "
        + "json:target/cucumber.json, "
        + "junit:target/cucumber-junit.xml")
public class RunCucumberTest {
}
```

`target/cucumber-report.html`

Para personas

Se abre con doble clic. Escenarios, pasos, tiempos y el error de cada fallo. Mañana se publica en GitHub Pages.

`target/cucumber.json`

Para herramientas

Xray, Allure o un tablero propio lo leen para armar su vista.

`target/cucumber-junit.xml`

Para la CI

El formato que GitHub Actions entiende: resumen de pruebas en el Pull Request.

CÓMO LEER EL ROJO

Cuatro preguntas, en este orden

1. **¿Qué escenario?** El nombre y la línea del `.feature`: *classpath:features/bugs.feature:21*.
2. **¿Qué paso?** El único con ✗. Los anteriores pasaron; los siguientes ni se ejecutaron.
3. **¿Aserción o excepción?** `AssertionError`: la API respondió otra cosa. `IllegalArgumentException`, `NullPointerException`: tu código no pudo ejecutarse.
4. **¿Qué envié y qué recibí?** El log que aparece encima del ✗. Con eso reproduces el caso en Postman o curl.

```
PS> mvn test "-Dcucumber.filter.tags=@bug"
[ERROR] Tests run: 31, Failures: 8, Errors: 0, Skipped: 23
[ERROR] BUILD FAILURE

PS> mvn test
Tests run: 31, Failures: 0, Errors: 0, Skipped: 8
[INFO] BUILD SUCCESS
```

SKIPPED: 8

Son los 8 escenarios `@bug` que el filtro por defecto deja fuera. El número no miente: hay 8 bugs conocidos esperando arreglo.

ROJO NO SIEMPRE ES MALO

Un escenario `@bug` en rojo **es la evidencia**. El día que pase a verde, el bug se corrigió: se le quita la etiqueta.

LO HACEMOS JUNTOS

Guion de la demo: refactorizar sin romper nada

1. **Línea base:** `mvn test` en verde. Contamos cuántas veces aparece `contentType` en los steps.
2. **Crear** `Especificaciones.base()` y usarla en un step. Correr: sigue verde.
3. **Extraer** `ReservaCliente`, inyectarlo por constructor y borrar el código repetido.
4. **Cambiar de ambiente** con `-DbaseUrl` a una URL que no existe y leer el log.
5. **Esquema negativo** de `validaciones.feature` y filtrar por `@negativo`.
6. **Abrir los tres reportes** de `target/` y mostrar dónde está el error de un escenario.

```
# 1 · ¿cuántas veces se repite?
PS> Get-ChildItem src\test\java -Recurse | Select-String
"contentType"
# 4 · otro ambiente
PS> mvn test "-Dcucumber.filter.tags=@smoke" "-
DbaseUrl=https://restful-booker.herokuapp.com/noexiste"
# 5 · solo los negativos
PS> mvn test "-Dcucumber.filter.tags=@negativo"
# 6 · el reporte
PS> start target\cucumber-report.html
```

LA REGLA DE ORO DEL REFACTOR

Verde antes, verde después, y en el medio pasos chicos. Si algo se pone rojo, el último cambio es el culpable.

DEL HALLAZGO AL REPORTE

Un bug = un escenario que falla + un párrafo que lo explica

features/bugs.feature (un ejemplo) — GHERKIN

```gherkin
# language: es
@bug @ejercicio5 @limpieza
Característica: Caza de bugs en Restful Booker

  Esquema del escenario: <bug> · Una reserva con <dato inválido> se rechaza
    Cuando intento crear una reserva con "<dato inválido>"
    Entonces el código de respuesta es 400

    Ejemplos:
      | bug    | dato inválido              |
      | BUG-01 | precio negativo            |
```

BUGS.md (un ejemplo) — MARKDOWN

```markdown
### BUG-01 · Acepta una reserva con precio negativo

- **Qué hice:** `POST /booking` con una reserva válida, pero `"totalprice": -100`.
- **Qué esperaba:** `400 Bad Request`. Un precio negativo no tiene sentido de negocio.
- **Qué pasó:** `200 OK` y la reserva queda guardada con `totalprice: -100`.
- **Severidad:** Alta. Permite reservas que "pagan" al huésped; afecta la facturación.
- **Escenario:** `BUG-01 · Una reserva con precio negativo se rechaza`.
```

ESCRIBE LO CORRECTO, NO LO QUE PASA

El *Entonces* dice **400** porque eso es lo que debería responder. Si escribes 200, tu prueba "pasa" y el bug queda escondido para siempre.

SEVERIDAD EN UNA LÍNEA

Alta: datos corruptos, dinero o seguridad. **Media**: el cliente se confunde. **Baja**: semántica o cosmética.

5
RETO

PENSAR COMO QA

Caza de bugs: la API esconde al menos cinco

50 min
Dificultad ●●●●●

OBJETIVO

Restful Booker tiene comportamientos que ninguna API de producción debería tener. Encuéntralos, escribe un escenario que **describa lo correcto** (y que hoy falle) y redacta el reporte del bug.

DÓNDE BUSCAR

1. Reglas de negocio: ¿acepta un precio negativo? ¿Una salida anterior a la llegada?
2. Validación: ¿qué pasa si falta un campo obligatorio en el cuerpo?
3. Seguridad: ¿qué código devuelve `/auth` con credenciales malas?
4. Semántica HTTP: ¿qué código devuelve un borrado exitoso?

CRITERIOS DE ACEPTACIÓN

1. Al menos **cinco** escenarios con `@bug`, cada uno con el resultado **correcto** en el *Entonces*.
2. `mvn test` queda en verde; `mvn test "-Dcucumber.filter.tags=@bug"` muestra los fallos.
3. Un `BUGS.md` con, por bug: título, qué hiciste, qué esperabas, qué pasó y severidad.
4. Lo de hoy aplicado: especificación reutilizable, URL por `-DbaseUrl`, log solo si falla, `@smoke` y `@regresion`.

Para subir de nivel

Busca más allá de la lista: ¿una fecha que no es fecha? ¿Un precio que no es número? ¿Borrar lo que no existe? Cada hallazgo propio suma.

Aprendes: diseño de pruebas negativas, valores límite, redactar bugs, etiquetar con `@bug` para no bloquear el pipeline.

LEER EL ROJO

Errores típicos de hoy y cómo salir de ellos

`DuplicateStepDefinition`

La misma frase dos veces

Al mover steps entre clases quedó el método en las dos. Cucumber no elige: borra uno.

`"<token>" literal`

Columna mal escrita

Si el encabezado de *Ejemplos* no coincide con el `<nombre>`, Cucumber envía el texto tal cual.

`Unknown lifecycle phase`

Comillas en PowerShell

`-Dcucumber.filter.tags=@smoke and not @bug` sin comillas se parte en tres. Envuelve todo el `-D…`.

`Skipped: 31 · BUILD SUCCESS`

Filtro que no coincide

Con `@Smoke` o `@regresión` no corre nada y Maven dice éxito. Las etiquetas distinguen mayúsculas y tildes.

`NullPointerException en un step`

El Contexto vino vacío

Un step usa `contexto.idReserva` y ningún paso anterior lo llenó. Falta un *Dado*.

`@bug en verde`

¿Se arregló o está mal escrito?

Si un escenario `@bug` pasa, o la API se corrigió o el *Entonces* describe el comportamiento con bug. Revísalo.

CANVIA

CIERRE Y TAREA

Un proyecto que crece sin dolor y un QA que sabe dónde mirar

Entrega

PR **feature/s3-<tu-nombre>** a `develop` con los escenarios `@bug`, el `BUGS.md` y la CI en verde.

Para practicar

Corre la regresión con `@regresion` y luego solo `@seguridad`. Compara tiempos y cantidad de escenarios.

Mañana

GitHub Actions paso a paso, el reporte publicado en GitHub Pages y el examen final sobre Canvia Market.

Documentación: **rest-assured.io** · **cucumber.io/docs**